
XpenseTrack / Cashalo

System Handover & Self-Hosting Guide

A complete, step-by-step guide for the receiving company's IT team to run the Expense, Approval, AP/AR, Payroll and BIR Form 2307 application entirely on its own infrastructure — database, backend API, and web frontend.

Application	Expense / AP-AR / Payroll management web app
Components	PostgreSQL database • Node.js API • React (Vite) web app
Document type	Turnover / deployment runbook
Prepared for	Receiving company IT / DevOps team

1. What this system is

The application has three independent parts. You will host all three.

1 · Database	2 · Backend (API)	3 · Frontend (Web app)
PostgreSQL. Stores all data (users, expenses, invoices, settings). Tables are created automatically on first boot.	Node.js / Express. Folder backend/. Serves the API at /api. Runs as a long-lived process.	React (Vite). Folder frontend/. A static site (HTML/JS/CSS) users open in a browser.

Data flow: the browser loads the **frontend**, which calls the **backend API** over HTTPS, which reads and writes the **PostgreSQL database**.

2. What to hand over (inventory)

Give the receiving team everything below. Items marked **critical** are required for the app to run at all.

A. Source code & documents

Item	Detail	Priority
Full source code	The complete project zip (this package) or access to the Git repository. Contains backend/ and frontend/.	Critical
BIR 2307 template	backend/src/templates/2307_template.pdf — must be present or 2307 generation fails. (Already inside the zip.)	Critical
This handover PDF	The document you are reading — the setup runbook.	High
Env templates	backend/.env.example and frontend/.env.example — annotated lists of every setting. (Inside the zip.)	High
Existing data (if any)	A PostgreSQL dump (pg_dump) if you are migrating live data from the current system.	If applicable

B. Credentials & secrets (send securely — never by plain email/chat)

- Database connection string (or let them create a fresh database — preferred).
- A new JWT_SECRET value (a long random string), or let them generate their own.
- Any third-party API keys they will actually use: email provider (MailerSend or Resend), OCR (OCR.space), Anthropic. **Rotate the Anthropic key first if the old one was ever exposed.**

C. Account / platform access (only if they take over hosting accounts)

- Repository access (GitHub) — add them as collaborators/owners.
- Hosting accounts for the backend and frontend, if you are transferring the existing ones.
- Domain / DNS management access, if the app uses a custom domain.

3. Prerequisites

- **Node.js 18 or newer** and **npm** (to build and run the app).
- A **PostgreSQL** database (a managed service such as Railway, Supabase, Neon, or AWS RDS is easiest).
- A place to run a **Node process** for the backend (Railway, Render, Fly.io, or a Linux VM).
- A place to serve a **static website** for the frontend (Vercel, Netlify, or Nginx / S3 + CDN).
- A domain name (optional but recommended).

4. Step 1 — Create the database

- Create an **empty** PostgreSQL database.
- Copy its connection string, in this form:

```
postgresql://USER:PASSWORD@HOST:5432/DBNAME
```

You do **not** create tables by hand — the backend creates them automatically on first start. If you are migrating existing data, restore your pg_dump into this database **before** starting the backend.

5. Step 2 — Deploy the backend (API)

Folder: backend/

5.1 Configure environment

Copy backend/.env.example to .env and fill it in (or set the same variables in your host's dashboard).
Minimum required: DATABASE_URL, JWT_SECRET, PORT.

Generate a strong JWT secret with:

```
openssl rand -hex 32
```

5.2 Install & start

```
cd backend
npm install
npm start      # syncs the DB schema, then starts the API
```

5.3 Create the first administrator (one time)

On a brand-new database there are no users yet. Run the seed once to create the first ADMIN:

```
npm run db:seed
```

It uses SEED_ADMIN_EMAIL and SEED_ADMIN_PASSWORD from your .env. The account is forced to change its password on first login. Every other user is created afterwards inside the app under **Settings** → **Users**.

Good to know

The API listens on PORT (default 3001) and serves routes under /api.

CORS is open to any origin, so the frontend can safely live on a different domain.

On Railway / Render: point the service at the backend/ folder, set the env vars in the dashboard, and use npm start as the start command.

6. Step 3 — Deploy the frontend (web app)

Folder: frontend/

- Set VITE_API_URL to the backend's public URL **including the trailing /api** — e.g. `https://api.yourcompany.com/api`.
- Build the static site:

```
cd frontend
npm install
npm run build      # outputs to frontend/dist
```

- **Vercel / Netlify**: set project root to frontend/, build command `npm run build`, output directory `dist`, and add the VITE_API_URL environment variable.
- **Nginx / Apache / S3**: upload the contents of `dist/` and enable SPA fallback (serve `index.html` for unknown routes).

Important

VITE_API_URL is baked in at **build time**. If you ever change the backend URL, you must rebuild and redeploy the frontend.

7. Step 4 — Domain, and first login

7.1 Domain / DNS (optional)

- Point e.g. `expenses.yourcompany.com` to the frontend host and `api.yourcompany.com` to the backend host.
- Set FRONTEND_URL in the backend env to the frontend URL so email links resolve correctly.

7.2 First login & configuration

- Open the frontend URL and log in with the seeded admin.
- Change the password when prompted.
- Go to **Settings** and configure: company name, logo, currency, categories, **ATC codes for BIR Form 2307**, approval flows, and the signatory details used on the 2307.
- Create the rest of the users under **Settings** → **Users**.

8. Administrator access (the super-admin account)

There is **no hidden or hard-coded super-admin** and no secret backdoor. The very first administrator is created by **you (the receiving team)** when you run the seed — so only you ever know its password.

8.1 How the first admin is set

Before running the seed, choose the admin's email and password in the backend `.env`:

```
SEED_ADMIN_EMAIL="admin@yourcompany.com"
SEED_ADMIN_PASSWORD=""
```

Then run the seed once:

```
cd backend
npm run db:seed
```

8.2 How you will know the credentials

- The seed **prints the admin email and password to the console/terminal** when it finishes — that is how you confirm them.
- The account is flagged **must-change-password**, so at first login the app forces a new password. After that, only the admin knows it.
- If you do **not** set the two variables above, the seed falls back to defaults `admin@company.com` / `ChangeMe123!` (also printed to the console, and still must be changed at first login).

What the seed prints

```
=====
✓ First ADMIN account created
  Email:    admin@yourcompany.com
  Password: ***** (must be changed on first login)
=====
```

8.3 Adding more admins

Once logged in, the first admin creates everyone else in **Settings** → **Users** and can set any user's role to **ADMIN**. You do not need the seed again.

8.4 If you get locked out (recovery)

- Set `BOOTSTRAP_ADMIN_EMAIL` in the backend env to an existing user's email.
- Redeploy / restart the backend once — on boot it promotes that user to ADMIN and re-activates it.
- Remove the variable afterwards so it doesn't run again.

Security

If you seed the admin **on behalf of the company**, share the temporary password through a **secure channel** (password manager or encrypted message) — never plain email or chat — and have them change it immediately.

Recommended: let the company set `SEED_ADMIN_PASSWORD` themselves so no one else ever sees it.

9. Updating the app later

- **Backend:** deploy new code — it re-syncs the database schema automatically on boot.
- **Frontend:** rebuild and redeploy the static site.
- Because the frontend is a PWA, users may need a **hard refresh** (Ctrl/Cmd+Shift+R) to see changes. If a change still won't appear, use the browser's **Clear site data** once.

10. Environment variable reference

Backend — required

Variable	Required?	What it is
DATABASE_URL	Yes	PostgreSQL connection string.
JWT_SECRET	Yes	Long random secret used to sign login tokens. Keep private; changing it logs everyone out.
PORT	Usually auto	Port the API listens on (default 3001). Many hosts set this for you.

Backend — first-admin seed (used by `npm run db:seed`)

Variable	Required?	What it is
SEED_ADMIN_EMAIL	For seed	Email of the first admin account.
SEED_ADMIN_PASSWORD	For seed	Temp password; must be changed on first login.
SEED_ADMIN_FIRSTNAME	Optional	First name of the admin (default 'System').
SEED_ADMIN_LASTNAME	Optional	Last name of the admin (default 'Administrator').
SEED_COMPANY_NAME	Optional	Initial company name shown in the app.

Backend — recommended / optional

Variable	Required?	What it is
FRONTEND_URL	Recommended	Public URL of the frontend; used in email links.
NODE_ENV	Recommended	Set to 'production' in production.
BOOTSTRAP_ADMIN_EMAIL	Recovery only	On boot, promotes these existing emails to ADMIN. Use only to recover a locked-out admin, then remove.
MAILERSEND_API_KEY / RESEND_API_KEY	Optional	Enable email notifications (pick one provider). App runs without it.
OCRSPACE_API_KEY / ANTHROPIC_API_KEY	Optional	Enable receipt OCR autofill. Rotate the Anthropic key if it was ever exposed.

Frontend — required

Variable	Required?	What it is
VITE_API_URL	Yes	Backend API URL, INCLUDING the trailing /api. Baked in at build time.

11. Troubleshooting (smooth setup)

Symptom	Likely cause & fix
Can't log in on a new database	You haven't seeded the first admin. Run <code>npm run db:seed</code> in the backend.
Frontend loads but every action fails / network errors	<code>VITE_API_URL</code> is wrong or missing the <code>/api</code> suffix. Fix it and rebuild the frontend.
Backend won't start / DB errors on boot	Check <code>DATABASE_URL</code> is correct and the DB is reachable from the backend host. Watch the boot logs — schema sync prints its status.
Everyone got logged out after a redeploy	<code>JWT_SECRET</code> changed. Keep it constant across deploys.
2307 PDF fails to generate	Confirm <code>backend/src/templates/2307_template.pdf</code> was deployed with the code.
New frontend changes not showing	PWA cache. Hard refresh (Ctrl/Cmd+Shift+R); if needed, Clear site data once.
No emails are sent	No email provider key set. Add a MailerSend or Resend key, or continue without email.
Locked out of the only admin account	Set <code>BOOTSTRAP_ADMIN_EMAIL</code> to that email, redeploy once, then remove the variable.

12. Security checklist

Please read before go-live

This codebase has had a **static access-control review**, not a **full penetration test**. Before relying on it for real money, commission an independent security review and run a limited pilot first.

- Use a strong, unique `JWT_SECRET`. Never commit `.env` files to source control.
- **Rotate the Anthropic API key** if the previous key was ever shared or screenshotted.
- Serve both frontend and backend over **HTTPS** only.
- Restrict database network access so only the backend can reach it.
- Review user roles and approval flows so no one approves their own submissions.
- Take regular database backups.

13. Important disclaimers

- **Not tax advice.** BIR Form 2307 ATC codes, rates, and computed figures must be verified by your accountant before issuing certificates. The default withholding base assumes VAT-inclusive amounts ($\text{amount} \div 1.12$) — adjust for non-VAT invoices.
- Test all money-related flows (approvals, exports, 2307, payments) in a staging environment before go-live.

14. Go-live checklist

✓	Complete each item before going live
■	PostgreSQL database created; connection string ready
■	Existing data restored (if migrating)
■	Backend .env configured (DATABASE_URL, JWT_SECRET, PORT)
■	Backend deployed and running; boot logs clean
■	First admin created via <code>npm run db:seed</code> ; password changed
■	Frontend VITE_API_URL set (ends in /api); site built and deployed
■	Domain / DNS pointed; FRONTEND_URL set on backend
■	Company settings, ATC codes, approval flows, and users configured
■	Anthropic key rotated; secrets stored securely; HTTPS enforced
■	Money flows (approval, export, 2307, payments) tested in staging
■	Database backup schedule in place

See `backend/.env.example` and `frontend/.env.example` in the source package for the full annotated list of every setting.